

Trevon Woods  
Sitaram Ayyagari  
A.I. at the Edge - ITAI - 3377  
01/25/2026

## Conceptual Report

Just to clarify, I have modified the number of steps from 7, as was originally shown in the assignment description, down to 5. This is because the first two steps from the assignment description are only necessary if I were to download Jupyter Notebooks locally on my computer, instead I used colab. With that being said, Step 1 was to “Load and Preprocess” the data downloaded from the MNIST dataset.

```
import tensorflow as tf
from tensorflow.keras.datasets import mnist
import numpy as np
import matplotlib.pyplot as plt

# Load MNIST dataset
(x_train, y_train), (x_test, y_test) = mnist.load_data()
# Normalize data (scale pixel values between 0 and 1)
x_train, x_test = x_train / 255.0, x_test / 255.0
# Show sample images |
plt.figure(figsize=(5,5))
for i in range(9):
    plt.subplot(3,3,i+1)
    plt.imshow(x_train[i], cmap="gray")
    plt.axis('off')
plt.show()
```

The loading was accomplished through the use of the `mnist.load_data()` function that is imported from Tensorflow’s vast collection of datasets. The preprocessing was accomplished through dividing the pixels of the images in the train and test data by 255. By doing this we were able to scale the pixels to a value between 0 and 1, essentially turning the image black & white. This is done so our training is less computationally intensive being that it would be easier to do calculations on smaller values rather than larger ones. Then we end off by using matplotlib to showcase 9 images from our preprocessed training dataset.

For Step 2, we create our neural network architecture by creating a variable and using the keras modules sequential function. This allows us to arrange tensorflow layers in a sequence that our input data will pass through. The first layer is a flatten layer, followed by 2 dense layers. The

first dense layer is the hidden layer in the architecture which has 128 neurons. The second dense layer has 10 neurons which are our 10 class outputs. We use the last layer to extrapolate the model's prediction on our input sample. Then we compile our model with the accuracy metric with which to track our models progress as well as how well our model makes predictions on the dataset. We also add a loss function which in this case is `sparse_categorical_crossentropy` and the Adam algorithm for optimization. The `sparse_categorical_crossentropy` is used to determine how far or close our models predictions are to calculate the loss, this is then used to update the models weights during backpropagation. Finally we start training our model, we call the `.fit()` function which is called with the training samples along with their labels, the number of epochs, and lastly the validation data. The training process occurs first by flattening the input tensor by multiplying the height and width dimensions to convert our rank 2 tensor into a rank 1 tensor. Then the model does computations on the input in the hidden layer from which we get out 10 output probabilities in the last layer. Lastly we save the model for use later using the `model.save()` function.

```
# Define model architecture
model = tf.keras.models.Sequential([
    # Input layer (flatten 28x28 pixels)
    tf.keras.layers.Flatten(input_shape=(28,28)),
    # Hidden layer
    tf.keras.layers.Dense(128, activation='relu'),
    # Output layer (10 classes)
    tf.keras.layers.Dense(10, activation='softmax')
])

# Compile model
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
# Train model
model.fit(x_train, y_train, epochs=5, validation_data=(x_test, y_test))
# Save trained model
model.save("mnist_model.h5")
print("Model training complete and saved as mnist_model.h5")
```

For Step 3, we convert our saved model to a TensorflowLite version. We first load our model using Keras's `load_model()` function and pass it the model file name. Then the model is converted using the `tf.lite.TFLiteConverter.from_keras_model()` passing in our model variable and calling `.convert()`. Finally, the model is saved with the new file name and `.tflite` file extension.

```

# Load trained model
model = tf.keras.models.load_model("mnist_model.h5")

# Convert to TensorFlow Lite
converter = tf.lite.TFLiteConverter.from_keras_model(model)

tflite_model = converter.convert() # Save the converted model
with open("mnist_model.tflite", "wb") as f:
    f.write(tflite_model)
    print("Model successfully converted and saved as mnist_model.tflite")

```

Step 4 was pretty straight forward, first we simply loaded the model again by calling the `tf.lite.interpreter()` function on the model path. The interpreter allows us to run a trained model efficiently on edge or mobile devices. Next, we reserve memory for all input and output tensors as well as intermediate activations between layers. This is done by calling the `.allocate_tensors()` function on our interpreter. Without this we would not be able to do inference. Lastly we showcase the input and output layer structures in our model.

```

# Load TensorFlow Lite model
interpreter = tf.lite.Interpreter(model_path="mnist_model.tflite")
interpreter.allocate_tensors()
# Get input and output tensor details
input_details = interpreter.get_input_details()
output_details = interpreter.get_output_details()
print("Input Details:", input_details)
print("Output Details:", output_details)

```

Finally we reach Step 5, the last and final step. In this step we run inference on our tensorflow lite model. We begin by converting our test sample pixels to float32 values as well as expand the dimension of our input tensor to match the expected input dimensions of our model. Now, before invoking our model to give us a prediction, we use `.set_tensor()` on our interpreter to load our input tensor into the memory buffer of our model at location `input_details[0]["index"]`. This allows our model to use the input image for inference. Now we are able to call `.invoke()` on our interpreter and extract the predictions using `.get_tensor()` on the memory location `output_details[0]["index"]` saving the value to our `output_data` variable. Lastly, we call the `argmax` function on our `output_data` to draw out the class with the highest probability. For good measure we show the image and its prediction paired with its ground truth label for comparison.

```
# Select a test image
test_image = x_test[0].astype(np.float32)

# Ensure data type matches model input
test_image = np.expand_dims(test_image, axis=0)

# Reshape for model input
# Set the input tensor
interpreter.set_tensor(input_details[0]['index'], test_image)

# Run inference
interpreter.invoke()

# Get the prediction
output_data = interpreter.get_tensor(output_details[0]['index'])
predicted_label = np.argmax(output_data)

# Display the image and prediction
plt.imshow(x_test[0], cmap="gray")
plt.title(f"Predicted Label: {predicted_label}, Actual Label: {y_test[0]}")
plt.show()
```